

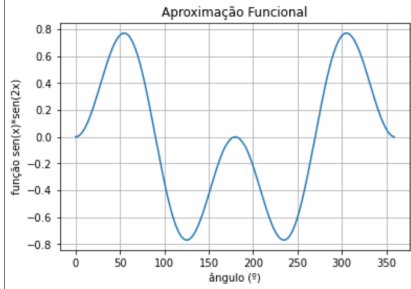
Redes Neurais Artificiais
Algoritmo *Backpropagation*

Aproximação Funcional

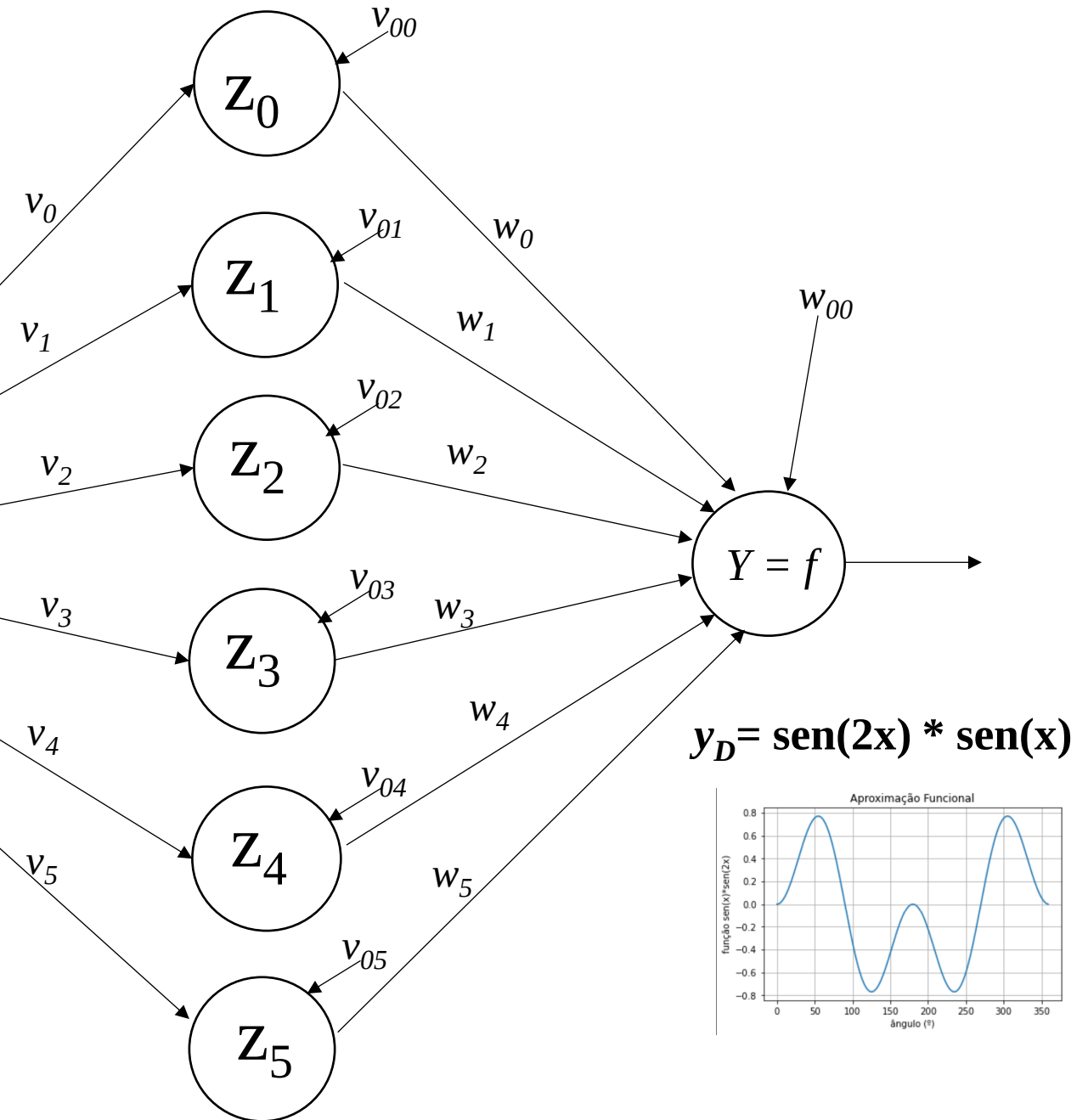
profº Mauricio Conceição Mario

Aproximação Funcional: utilização da Rede Neural para reproduzir na saída o padrão apresentado na entrada.

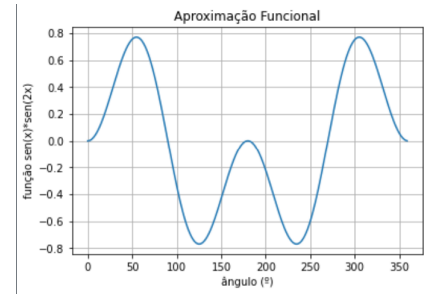
$$f(x) = \sin(2x) * \sin(x)$$



Arquitetura de rede proposta: Rede Neural utilizando algoritmo *Backpropagation* com uma entrada, uma camada intermediária com seis neurônios e função de ativação *sigmóide bipolar*, com atualização dos pesos em função do erro simples.



$$y_D = \sin(2x) * \sin(x)$$



(baseado em experimento proposto pelo profº Keiji Yamanaka - 2000)

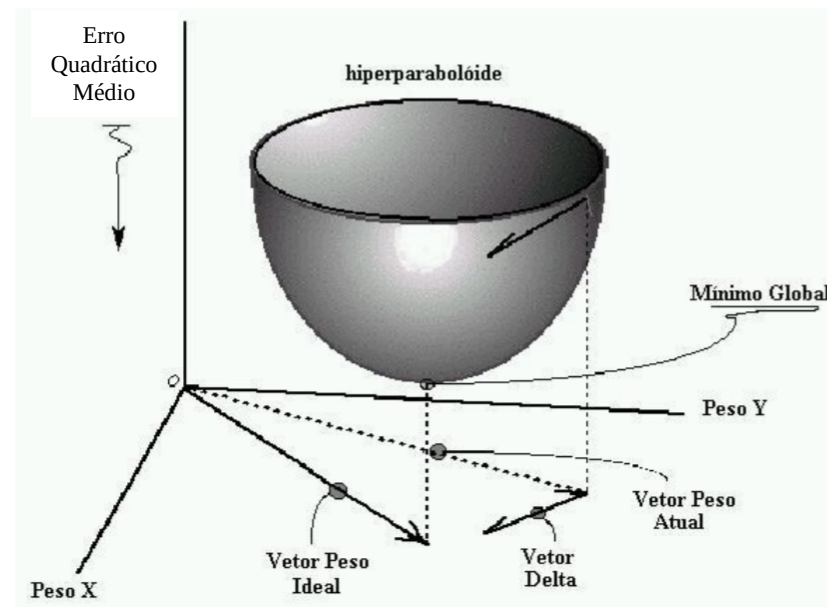
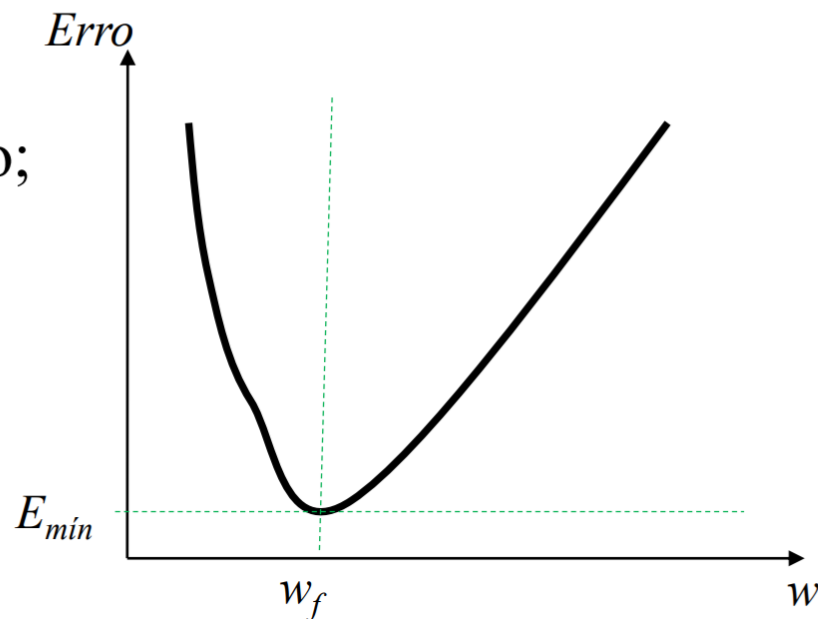
Aprendizado: atualização dos pesos

- O processo de atualização dos pesos pode ser chamado de convergência. Uma rede converge seus pesos para determinados valores quando o “**erro global**” diminui a cada ciclo de treinamento.
- O “**erro global**” pode ser entendido como a diferença (ou diferença média quadrática) entre saída(s) desejada(s) e saída(s) calculada(s). Sendo o “**erro global**” denotado por “Erro” e sendo “ w ” os valores que o conjunto de pesos que a rede pode assumir, tem-se um gráfico característico do “**erro global**” em função de “ w ”:

Onde:

E_{min} = erro mínimo;

w_f = vetor de pesos para o erro mínimo.



<https://sites.icmc.usp.br/andre/research/neural/>

Aprendizado: atualização dos pesos →

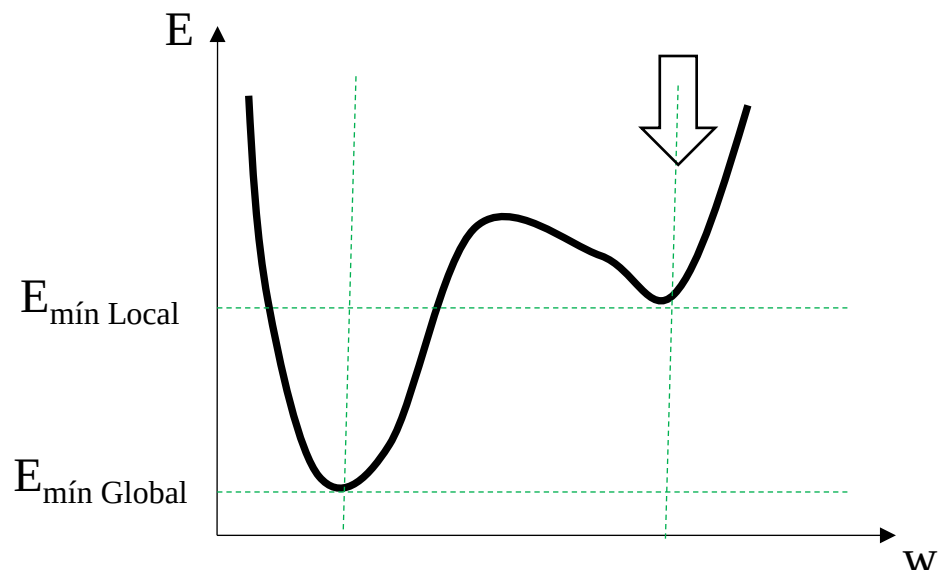
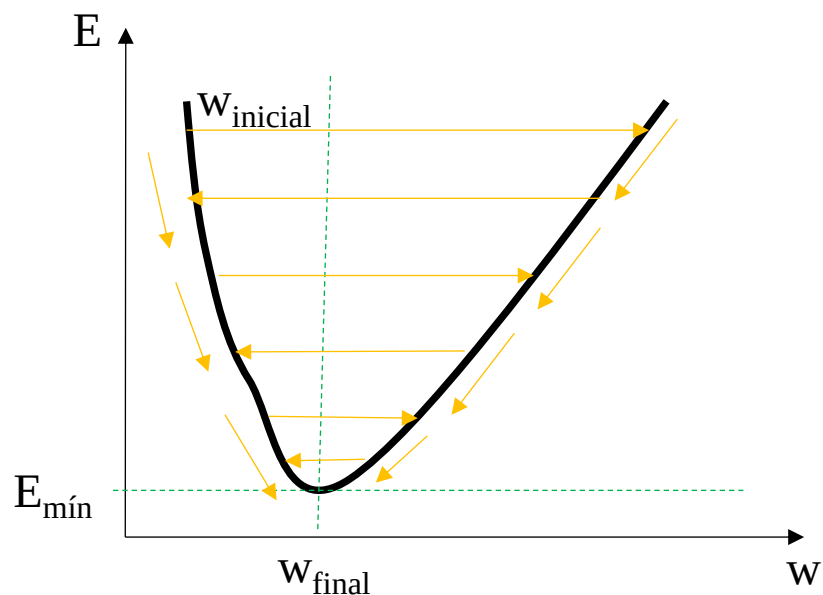
- O processo de descida do gradiente direciona o valor do peso em direção do mínimo erro.

$$E = \frac{1}{2} (Yd - Y)^2$$

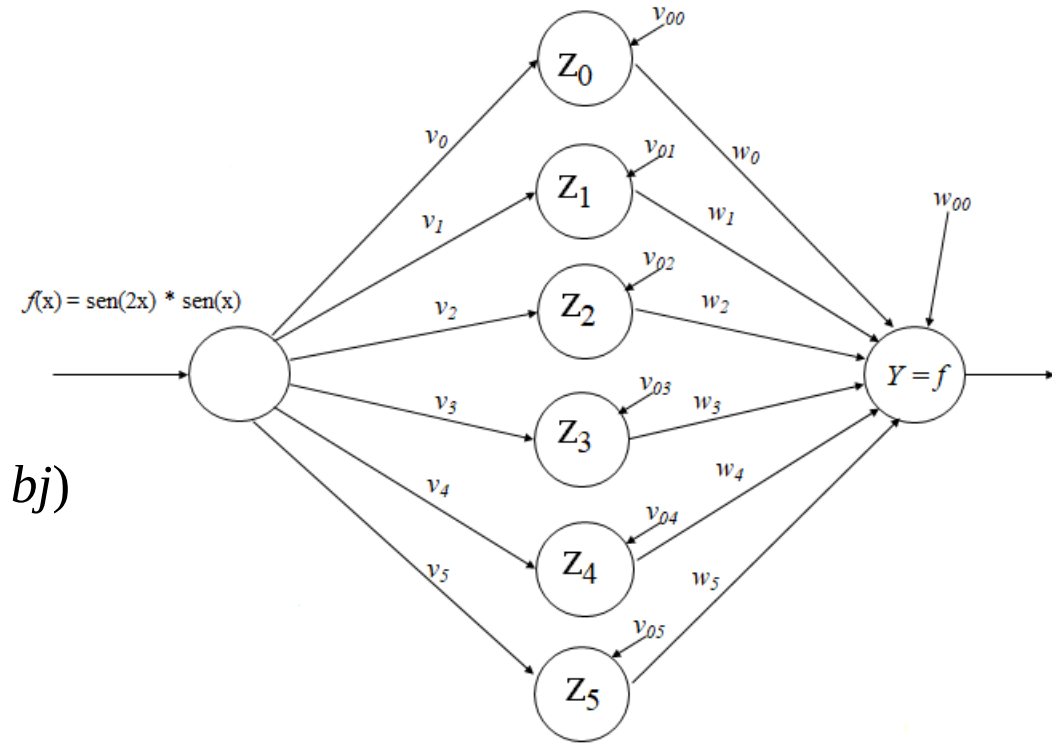
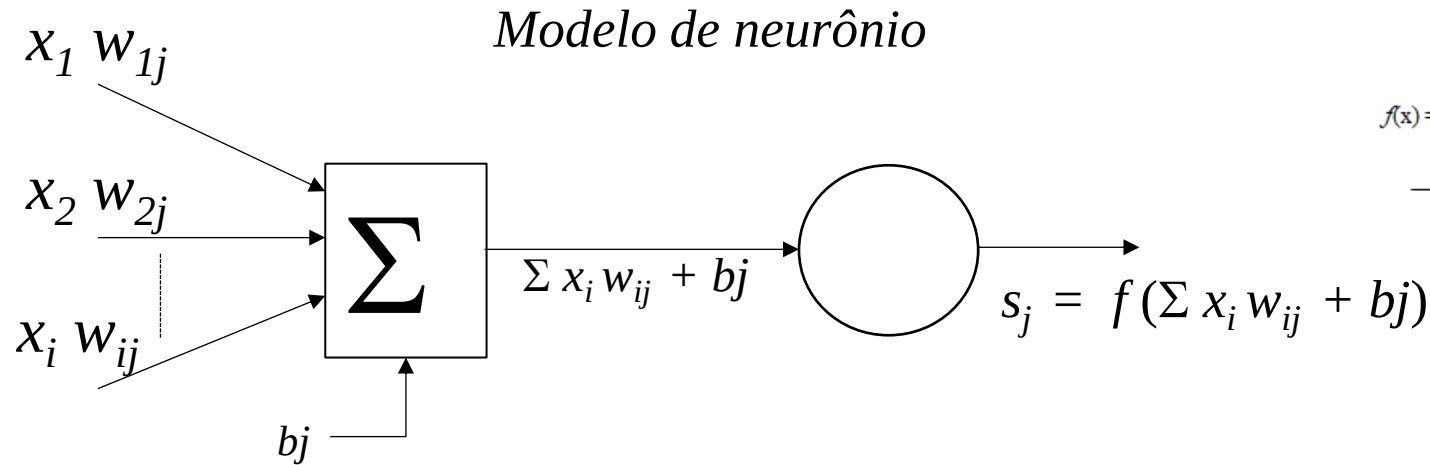
$$\Delta w_i \propto - \partial E / \partial w_i$$

$$\text{e } w_i(n + 1) = w_i(n) + \eta e x_i(n)$$

Dependendo da natureza da aplicação, a rede pode convergir para um valor de erro que se denomina de **erro mínimo local**, que não corresponde ao erro mínimo da rede (então denominado de **erro mínimo global**).



Backpropagation (Retropropagação)



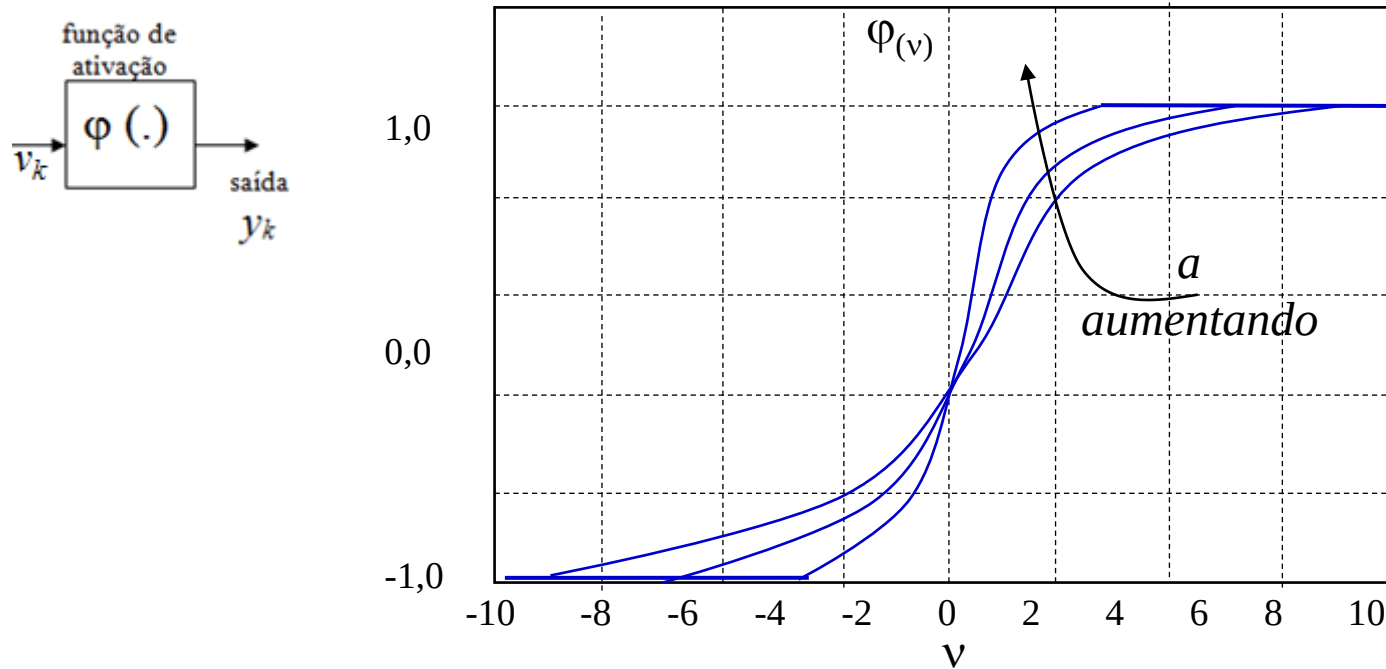
- **Características:** - a função de ativação deve ser uma função não-linear diferenciável em todos os pontos;
- Algoritmo de aprendizado supervisionado;
- Topologia de rede: permite n camadas internas com retropropagação de ajustes dos pesos;
- Processo de aprendizado: minimização do erro pelo método do gradiente descendente:

$$\Delta w_{ij} = -\alpha \frac{\partial E}{\partial w_{ij}} \quad \text{onde} \quad E_j = \frac{1}{2} (t - s_j)^2 \quad \text{ou} \quad E_j = (t - s_j)$$

(t : saída desejada e s_j = saída obtida)

(notas de aula do profº Keiji Yamanaka - 2000)

Redes Neurais Artificiais – funções de ativação



3. Função Sigmóide bipolar:

$$\varphi(v) = \frac{2}{1 + e^{-av}} - 1$$

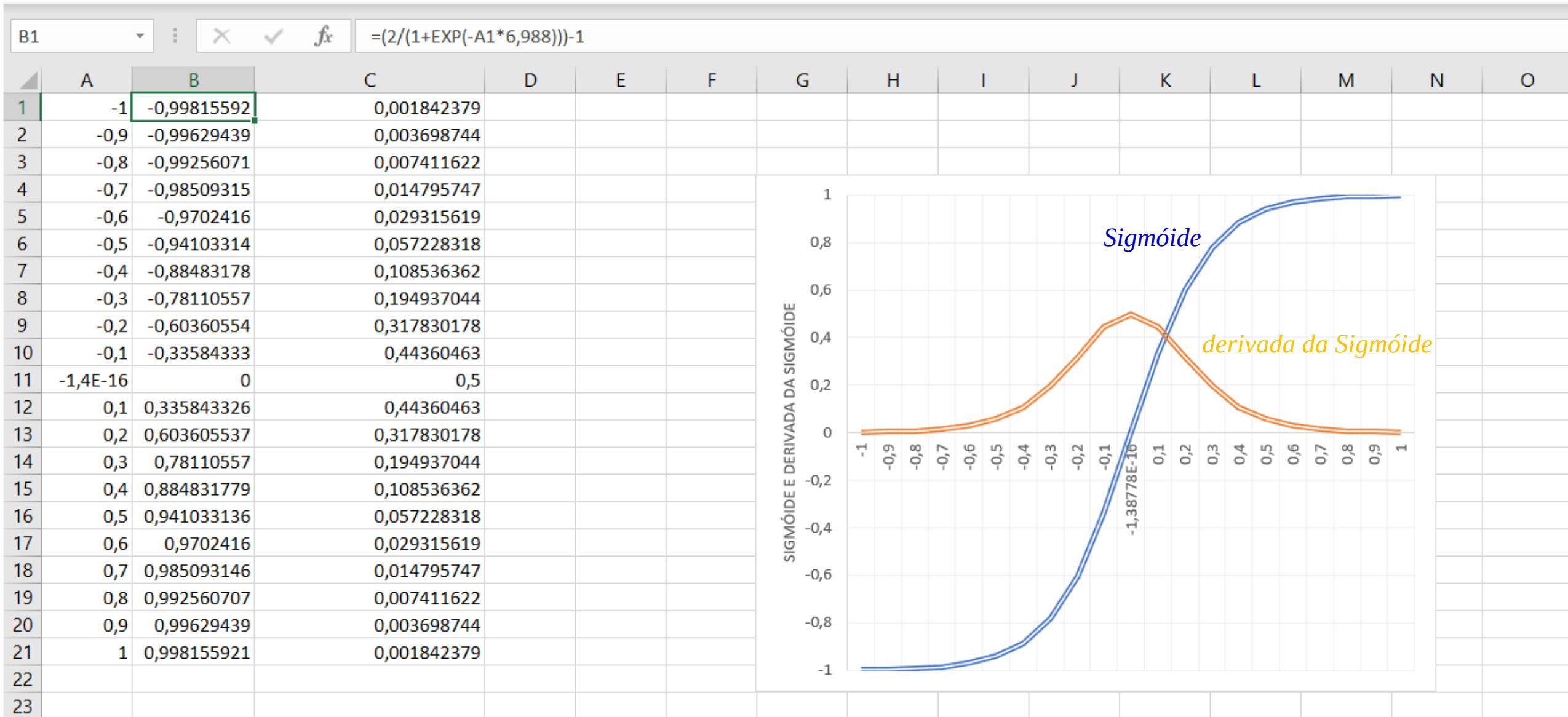
→ derivada

$$\varphi(v)' = 0.5 * (1 + \varphi(v)) * (1 - \varphi(v))$$

Onde a é o parâmetro de inclinação da função sigmóide.

(Haykin, 2001).

Sigmóide Bipolar e sua derivada



Algoritmo *Backpropagation*

1. Soma da junção aditiva nos neurônios da camada intermediária Z0 e Z1:

$$Z(x)_{[i]} = V0_{[i]} + v[i] * f(x)$$

2. Cálculo das funções de ativação sigmóide nos neurônios da camada intermediária:

$$Zf(x)_{[i]} = \frac{2}{1 + e^{-Z(x)[i]}} - 1$$

3. Cálculo da derivada da função de ativação:

$$Zff(x)_{[i]} = \frac{1}{2} * [(1 + Zf(x)[i]) * (1 - Zf(x)[i])]$$

4. Atualização de $Y(x)$:

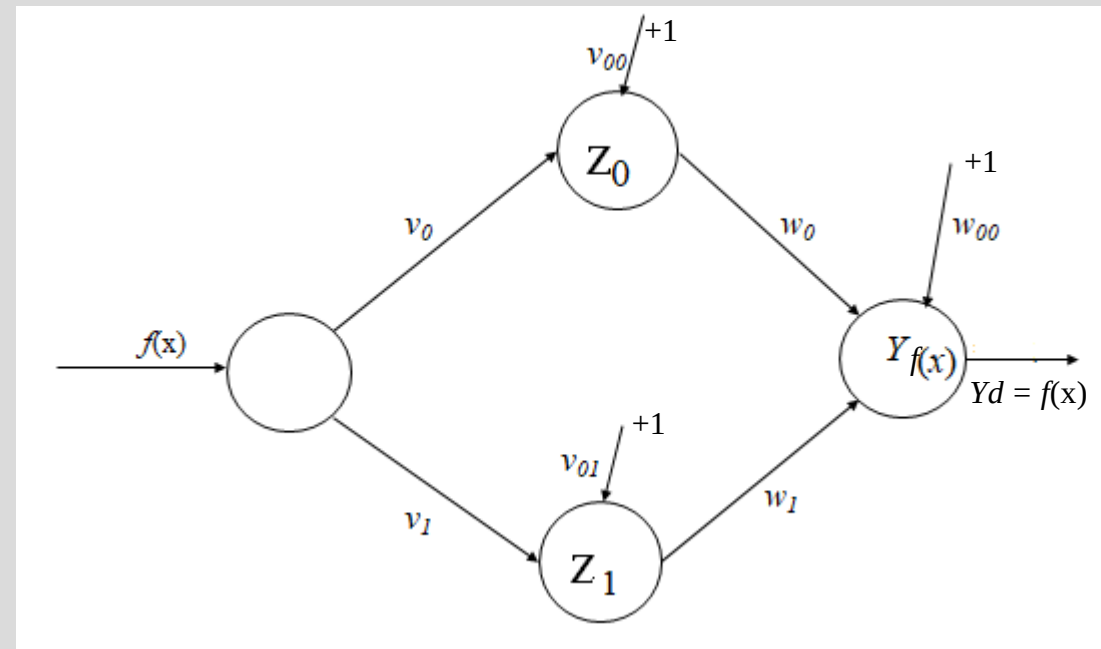
$$Y(x) = w_{00} + Zf[0] * w_0 + Zf[1] * w_1$$

5. Cálculo da função de ativação do neurônio na saída:

$$Yf[x] = \frac{2}{1 + e^{-Y(x)}} - 1$$

6. Cálculo do erro:

$$erro(x) = f(x) - Yf(x)$$



7. Cálculo da derivada da função de ativação na saída:

$$Yff(x) = \frac{1}{2} * [(1 + Yf(x)) * (1 - Yf(x))]$$

Algoritmo Backpropagation

Retropropagação

8. Cálculo do erro atualizado: α = taxa de aprendizagem

$$\text{erro_atualizado}(x) = \alpha * \text{erro}(x) * Y f f(x)$$

Atualização dos Δ e dos pesos das sinapses v e w

9. Atualização de Δw_{00} :

$$\Delta w_{00} = \alpha * \text{erro}(x)$$

10. Atualização de w_{00} :

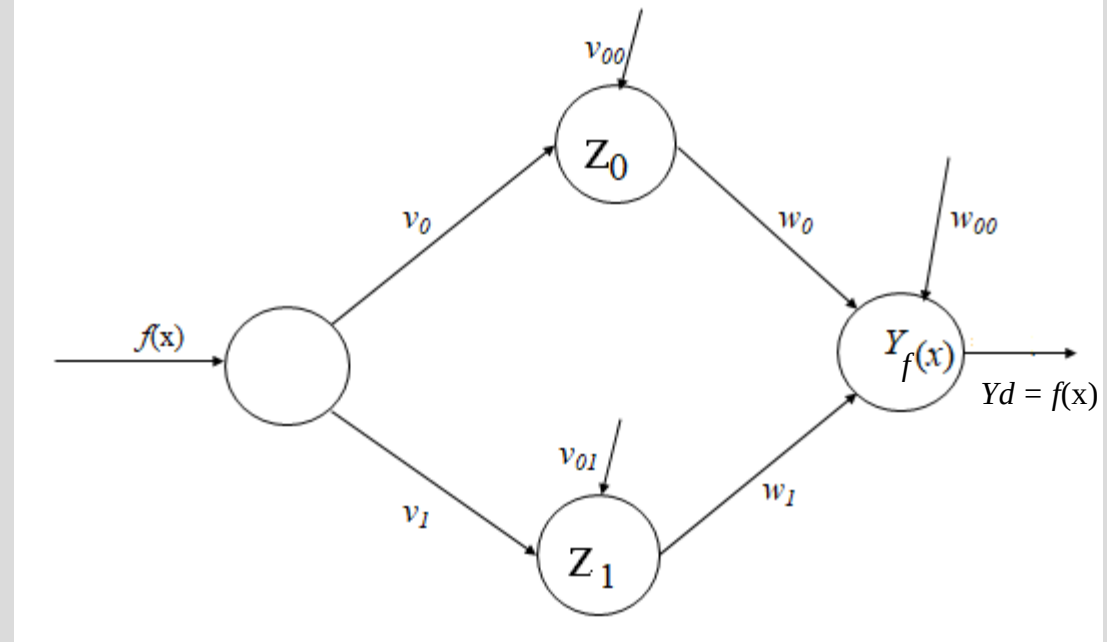
$$w_{00(n+1)} = w_{00(n)} + \Delta w_{00}$$

11. Atualização de $\Delta w_{[i]}$:

$$\Delta w_{[i]} = \text{erro_atualizado}(x) * Z f[i]$$

12. Atualização de $w_{[i]}$:

$$w_{[i](n+1)} = w_{[i](n)} + \Delta w_{[i]}$$



Atualização dos Δ e dos pesos das sinapses v e w

13. Atualização de $\Delta v_{0[i]}$:

$$\Delta v_{0[i]} = \alpha * \text{erro}(x) * Z f[i]$$

14. Atualização de $v_{0[i]}$:

$$v_{0[i](n+1)} = v_{0[i](n)} + \Delta v_{0[i]}$$

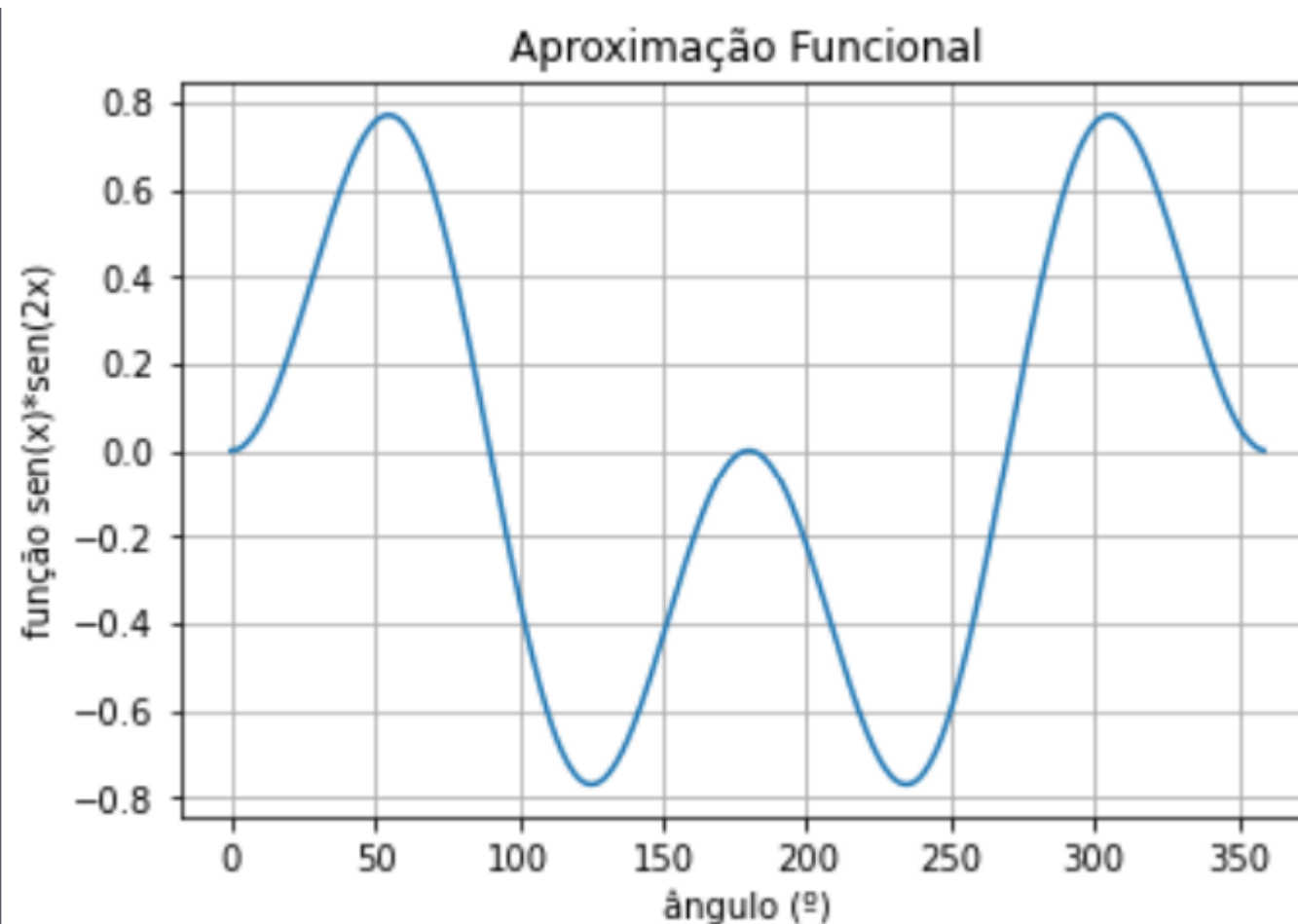
15. Atualização de $\Delta v_{[i]}$:

$$\Delta v_{[i]} = \Delta w_{[i]} * w_{[i]} * Z f f[i] * f[x]$$

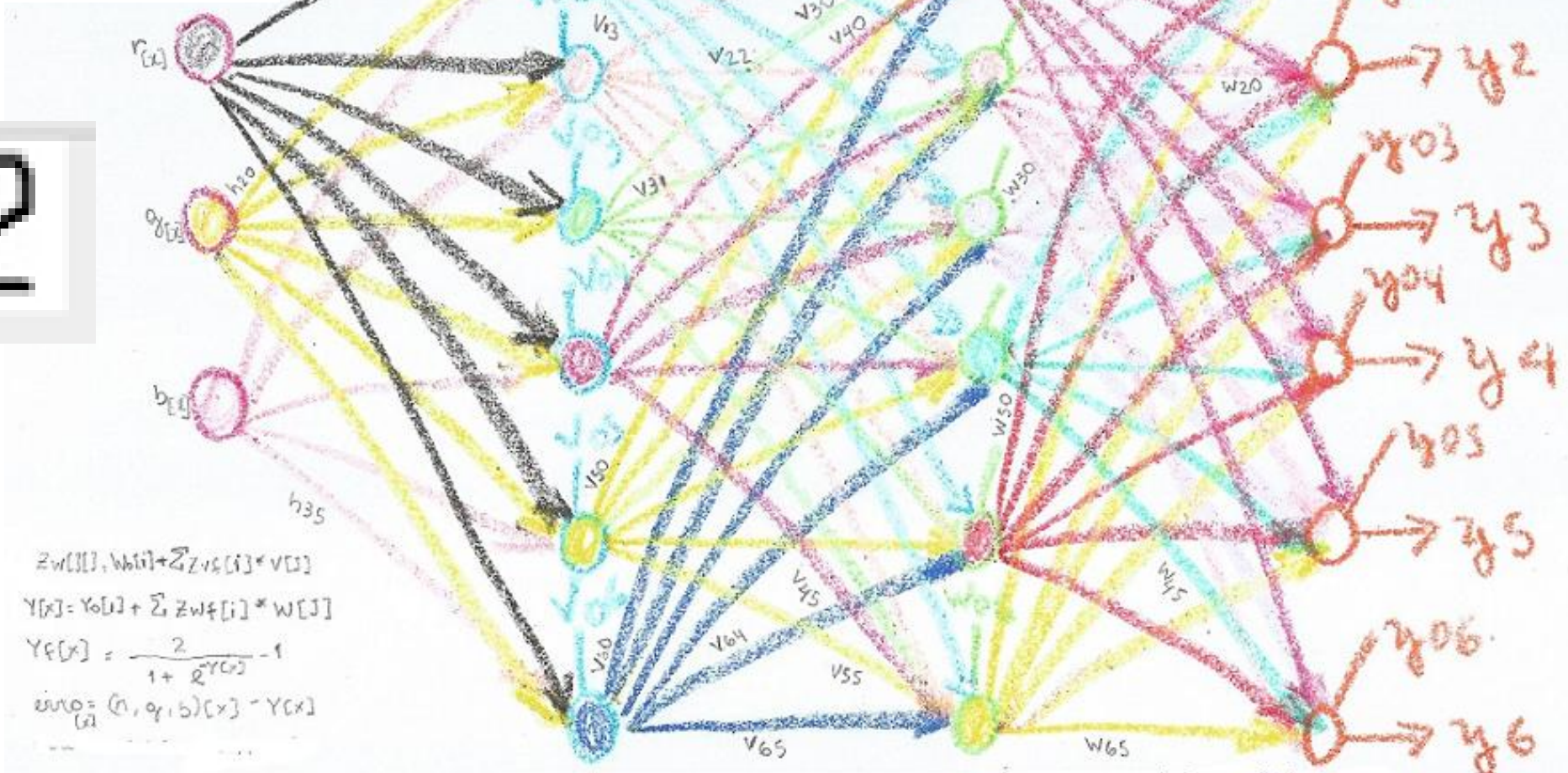
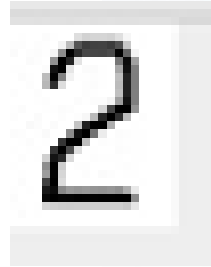
16. Atualização de $v_{[i]}$:

$$v_{[i](n+1)} = v_{[i](n)} + \Delta v_{[i]}$$

Aproximação funcional: função desejada na saída da rede, y_D é igual a função na entrada da rede $\rightarrow f(x) = \text{sen}(2x) * \text{sen}(x)$



Rede Neural Artificial
com duas camadas
intermediárias utilizada
para reconhecimento de
padrões de imagens.



$\Delta w_{ij} = \alpha \cdot y_j \cdot y_i \cdot [\text{somatória } w_{ij} \cdot \text{erroj}]$ - sinapse em camada intermediária
 $\Delta w_{ij} = \alpha \cdot y_j \cdot y_i \cdot \text{erroj}$ - sinapse ligada a neurônio na saída

```
5  @author: cmari
6  """
7  from linear_algebra import dot
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import math #pg 56 - Summerfield
11 import math, random
12
13
14 def sigmoid(t):
15     return ((2 / (1 + math.exp(-t)))-1)
16
17 def neurônio_MCP(pesos, entradas):
18     uk = dot(pesos, entradas)
19     return sigmoid(uk)
20
21 def feed_forward(rede_xor, vetor_entrada):
22     """recebe uma rede neural (representada como uma lista de listas de listas de pesos)
23     e retorna a saída da propagação direta da entrada"""
24
25     vetor_saída = []
26
27     for ponteiro in rede_xor:
28
29         entrada_com_bias = vetor_entrada + [1] # adiciona bias ao vetor de entrada
30         saída = [neurônio_MCP(neurônio, entrada_com_bias) # calcula a saída
31                 for neurônio in ponteiro] # para a camada de rede
32         vetor_saída.append(saída) # adiciona ao vetor de saída
33
34         # a entrada para a próxima camada é a saída desta etapa
35         vetor_entrada = saída
36
37     return vetor_saída
38
```


→ função *backpropagation* recebe os pesos da rede, entrada e saída desejada.

→ invoca função *feed_forward* para produzir uma saída *sigmóide* em cada neurônio da Rede Neural: cada neurônio tem uma saída computada.

→ calcula o *delta* para a camada de saída, considerando a derivada da *sigmóide*, erro e taxa aprendizagem.

→ faz a atualização dos pesos sinápticos para a camada de saída.

→ calcula o *delta* para a camada intermediária.

→ faz a atualização dos pesos sinápticos para a camada intermediária.

Algoritmo Backpropagation

```
39
40 alpha = 0.08
41 def backpropagation(rede_neural, vetor_entrada, vetor_saida):
42     # feed_forward calcula a saída dos neurônios usando sigmóide
43     saídas_intermediárias, saídas_neurônios = feed_forward(rede_neural, vetor_entrada)
44     # 0.5 * alpha * (1 + output) * (1 - output) cálculo da derivada da sigmóide
45     deltas_saida = [(0.5 * (1 + saída) * (1 - saída)) * (saída - vetor_saida[i]) * alpha
46                     for i, saída in enumerate(saídas_neurônios)]
47
48     # ajuste dos pesos sinápticos para camada de saída (rede_neural[-1])
49     for i, neurônio_saida in enumerate(rede_neural[-1]):
50         for j, saída_intermediária in enumerate(saídas_intermediárias + [1]):
51             neurônio_saida[j] -= deltas_saida[i] * saída_intermediária
52
53     # 0.5 * alpha * (1 + output) * (1 - output) cálculo da derivada da sigmóide
54     # retro-propagação do erro para camadas intermediárias
55     deltas_intermediários = [0.5 * alpha * (1 + saída_intermediária) * (1 - saída_intermediária) *
56                              dot(deltas_saida, [n[i] for n in rede_neural[-1]])
57                              for i, saída_intermediária in enumerate(saídas_intermediárias)]
58
59     # ajuste dos pesos sinápticos para camadas intermediárias (network[0])
60     for i, neurônio_intermediário in enumerate(rede_neural[0]):
61         for j, input in enumerate(vetor_entrada + [1]):
62             neurônio_intermediário[j] -= deltas_intermediários[i] * input
63
```

```
64
65 def seno(x): # função a ser aproximada pela Rede Neural
66     seno = [(math.sin(math.pi/180*x)*math.sin(2*math.pi/180*x))] #seno é uma lista
67     # [(0.8+(math.sin(math.pi/180*x)*math.sin(2*math.pi/180*x)))*0.5]
68     return [seno]
69
```

```
70 def teste(x):
71     #seno = [i for i in range(360)]
72     #for x in range(360):
73     teste = [math.cos(math.pi/180*x)*math.sin(2*math.pi/180*x)]
74     return [teste]
```

→ função da aproximação funcional $f(x) = \text{sen}(2x) * \text{sen}(x)$

função *enumerate* desmembra os elementos de *network* para a variável *output_neuron*, correspondentes aos pesos sinápticos do segmento de saída da rede atualizados.

→ função de entrada de teste não treinada para a rede neural:
 $f(x) = \cos(x) * \text{sen}(2x)$

```
def predict(entrada_teste):
    return feed_forward(rede_neural, entrada_teste)[-1]
```

→ função `predict()` é invocada para o teste da Rede Neural, ou seja, o sinal é aplicado no conjunto de pesos após o treinamento, e flui até a saída da rede.

```
entrada = []
saída_desejada = []
for x in range(360):
    seno_a = seno(x)
```

```
#TREINAMENTO DA REDE NEURAL
```

```
random.seed(0) # valores iniciais de pesos sinápticos
neurônios_entrada = 1 # dimensão do vetor de entrada
neurônios_intermediários = 6 # número de neurônios na camada intermediária
neurônios_saída = 1 # dimensão da camada de saída = 1 neurônio
```

```
"""
```

```
inserindo manualmente os vetores relativos à camada intermediária e à saída da Rede Neural
```

```
hidden_layer = [[-0.085, -0.09], [-0.033, -0.08], [-0.074, -0.063], [-0.075, -0.065], [-0.088, -0.076], [-0.077, -0.072]]
```

```
output_layer = [[0.082, -0.09, 0.064, -0.08, 0.084, -0.075, 0.099]] """
```

```
# cada neurônio da camada intermediária tem um peso sináptico associado à entrada
# e adicionando o peso do bias
```

```
camada_oculta = [[random.random() for __ in range(neurônios_entrada + 1)]
                  for __ in range(neurônios_intermediários)]
```

```
#print(hidden_layer)
```

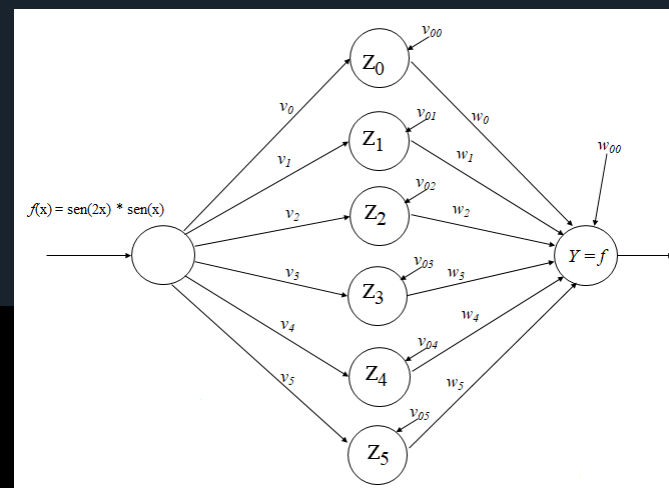
```
# neurônio de saída tem um peso sináptico associado a cada neurônio da camada intermediária
# e adicionando o peso do bias
```

```
camada_saída = [[random.random() for __ in range(neurônios_intermediários + 1)]
                 for __ in range(neurônios_saída)]
```

```
# a rede inicializa com pesos sinápticos randômicos
```

```
rede_neural = [camada_oculta, camada_saída]
```

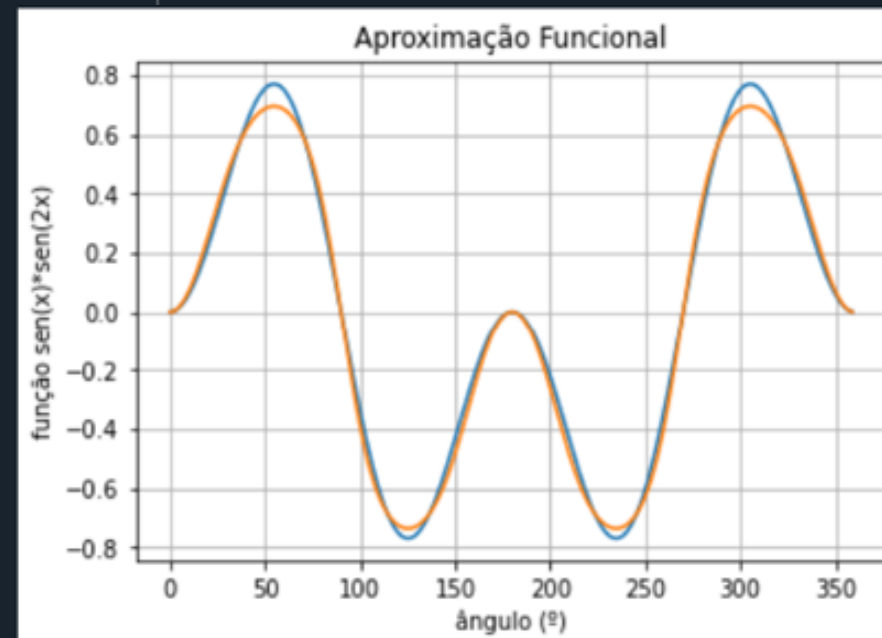
→ valores randômicos para os valores iniciais dos pesos sinápticos, gerados para as camadas oculta e de saída.



```

109 for _ in range(300): #número de ciclos da treinamento
110     for x in range(360):
111         entrada = seno(x)
112         saída_desejada = seno(x)
113         for vetor_entrada, vetor_saída in zip(entrada, saída_desejada):
114             backpropagation(rede_neural, vetor_entrada, vetor_saída)
115
116     #TREINAMENTO DA REDE NEURAL
117
118     # formatação do gráfico
119     fig, ax = plt.subplots()
120     ax.set(xlabel='ângulo (°)', ylabel='função sen(x)*sen(2x)',
121           title='Aproximação Funcional')
122     ax.grid()
123     t = np.arange(0, 360, 1)
124
125     # teste da rede através de predict( )
126     saída = []
127     for x in range(360):
128         entrada_treinada = seno(x)
129         entrada_teste = seno(x)
130         for vetor_entrada, vetor_teste in zip(entrada_treinada, entrada_teste):
131             sinal_saida = predict(vetor_teste)
132             saída.extend(sinal_saida)
133
134     entrada = []
135     for x in range(360):
136         entrada += seno(x) # criando o arranjo da função de entrada para o gráfico
137     ax.plot(t, entrada )
138     ax.plot(t, saída)
139     plt.show()
140
141     print ("camada intermediária", camada_oculta )
142     print ("camada saída", camada_saída )

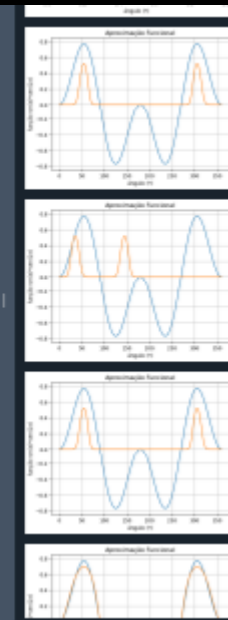
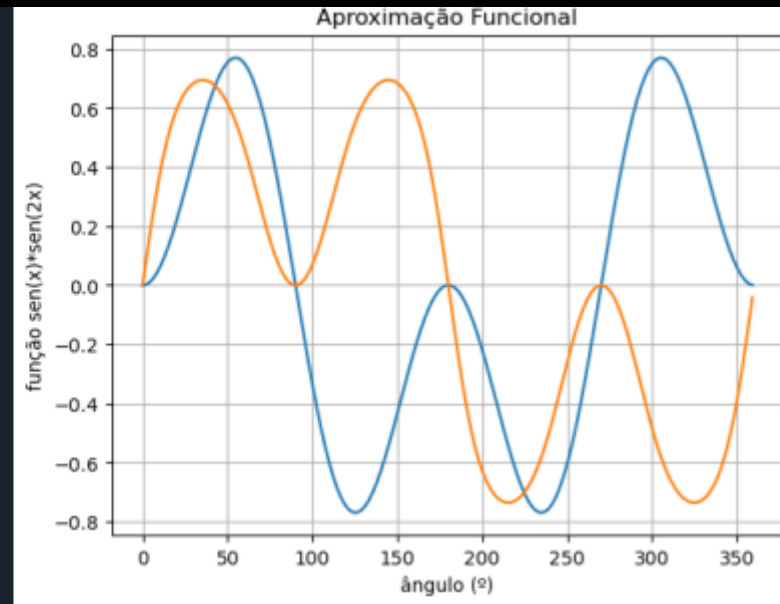
```



```

109 for _ in range(300): #número de ciclos da treinamento
110     for x in range(360):
111         entrada = seno(x)
112         saída_desejada = seno(x)
113         for vetor_entrada, vetor_saída in zip(entrada, saída_desejada):
114             backpropagation(rede_neural, vetor_entrada, vetor_saída)
115
116     #TREINAMENTO DA REDE NEURAL
117
118     # formatação do gráfico
119     fig, ax = plt.subplots()
120     ax.set(xlabel='ângulo (°)', ylabel='função sen(x)*sen(2x)',
121           title='Aproximação Funcional')
122     ax.grid()
123     t = np.arange(0, 360, 1)
124
125     # teste da rede através de predict( )
126     saída = []
127     for x in range(360):
128         entrada_treinada = seno(x)
129         entrada_teste = teste(x)
130         for vetor_entrada, vetor_teste in zip(entrada_treinada, entrada_teste):
131             sinal_saída = predict(vetor_teste)
132             saída.extend(sinal_saída)
133
134     entrada = []
135     for x in range(360):
136         entrada += seno(x) # criando o arranjo da função de entrada para o gráfico
137     ax.plot(t, entrada )
138     ax.plot(t, saída)
139     plt.show()
140
141     print ("camada intermediária", camada_oculta )
142     print ("camada saída", camada_saída )

```

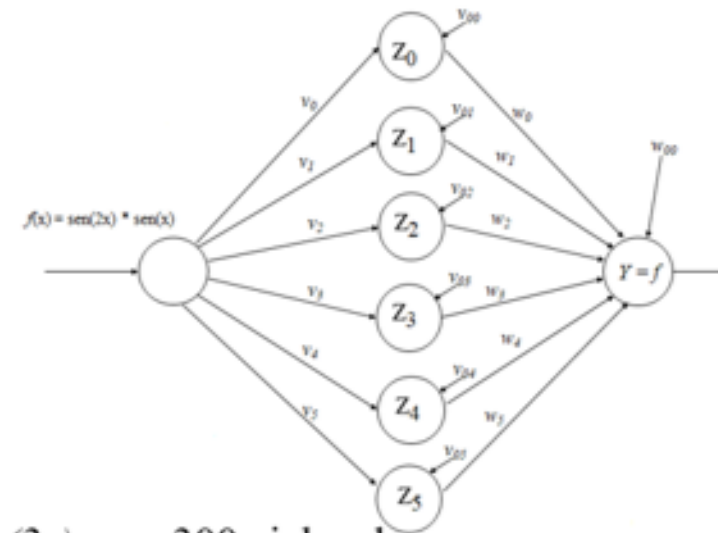
Ajuda Explorador de variáveis Figuras Arquivos



Console 1/A X

```
In [8]: runfile('D:/arquivos python/Backpropagation_aproximação_funcional.py', wdir='D:/arquivos python')
camada intermediária [[0.8316653739793877, 0.7329050252081365], [0.525967423866551, 0.17441528043000876], [0.5741673239635845, 0.335797171895594], [0.8083944325652166, 0.23227156718759506], [0.5443884812272576, 0.4999396190980019], [0.9293839135481534, 0.3288285099918338]]
camada saída [[0.5159123008913149, 1.3385546898770184, 1.106536827253515, 1.1610797917363482, 1.1431445370972968, 1.7584206654429964, -1.1827560941405226]]
```

→ *Aproximação Funcional*
utilizando algoritmo *Backpropagation*



- Fazer a aproximação funcional de $f(x) = \sin(x) * \sin(2x)$ para 300 ciclos de treinamento e 6 neurônios na camada intermediária, com o valor de $\alpha = 0.08$;
 - Variar alternadamente os parâmetros ciclos de treinamento, taxa de aprendizagem e número de neurônios na camada intermediária da rede e concluir qual a influência de cada um deles na performance da aproximação funcional.
- Inserir no teste de generalização (função $predict()$) o sinal de teste $\cos(x) * \sin(2x)$; justificar o resultado gráfico.

Referências Bibliográficas

- Braga AP, Carvalho APLF, Ludermir TB. *Redes Neurais Artificiais: teoria e aplicações*. Livros Técnicos e Científicos, Rio de Janeiro – RJ; 2007.
- Haykin S. *Neural Networks – A Comprehensive Foundation*. Prentice-Hall; 1994.
- Haykin S. *Redes Neurais – Princípios e prática*. 2a ed.. Porto Alegre: Bookman; 2001.
- Hebb DO. *The Organization of Behavior*. John Wiley & Sons; 1949.
- Heckerman D. *Probabilistic Similarity Networks*. MIT Press, Cambridge, Massachussets; 1991.
- Hopfield JJ. *Neurons with graded response have collective computational properties like those of two-state neurons*. Proceedings of the National Academy of Sciences of the United States of America, 79, 2554-2558; 1982.
- Minsky, M.; Papert, P.. *Perceptrons: An introduction to computational geometry*. MIT Press, Massachussets, 1969.

Referências Bibliográficas

- Mario MC. *Proposta de Aplicação das Redes Neurais Artificiais Paraconsistentes como Classificador de Sinais Utilizando Aproximação Funcional*. Univ. Federal de Uberlândia, Dissertação de Mestrado, Uberlândia; 2003.
- McCarthy J. *Programs with common sense*. In Proceedings of the Symposium on Mechanisation of Thought Processes, Vol. 1, pp. 77-84, London. Her Majesty's Stationery Office; 1958.
- McCulloch W, Pitts W. *A logical calculus of the ideas immanent in nervous activity*. Bulletin of Mathematical Biophysics 5, 115-133; 1943.
- Rosenblatt F. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan, Chicago; 1962.
- Rumelhart DE, McClelland JL. *Parallel Distributed Processing*. MIT Press, Cambridge, Massachusetts; 1986.
- Turing A. *Computing machinery and intelligence*. Mind, 59, 433-460; 1950.

Referências Bibliográficas

- Medeiros, L.F.. *Redes Neurais em Delphi*. Editora Visual Books. Florianópolis, 2003.
- Cybenko, G.. *Continuous valued neural networks with two hidden layers are sufficient*, Technical report, Department of Computer Science, Tufts University, 1988.
- Cybenko, G.. *Approximation by superpositions of sigmoid function*, *Mathematics of Control, Signals and Systems*, 2:303-314, 1989.